PYGAME CHEAT SHEET

A quick syntax reference for beginner game developers • Python 3

1. Setup & create a game window

SYNTAX / EXAMPLE	WHAT IT DOES
`pip install pygame`	Install Pygame in your Python environment (run in Terminal / Command Prompt).
`import pygame` `pygame.init()`	Import Pygame and initialize its modules.
`screen = pygame.display.set_mode((800, 600))` `pygame.display.set_caption("My Game")`	Create an 800 × 600 window and set its title.
`clock = pygame.time.Clock()` `running = True`	Create a clock to control frame rate and a flag for the game loop.

`WHITE = (255, 255, 255)`
`BLACK = (0, 0, 0)`
`RED = (255, 0, 0)`
	Define colors using RGB values (each channel is 0–255).

2. The game loop

SYNTAX / EXAMPLE	WHAT IT DOES

`while running:`
`  for event in pygame.event.get():`
`  if event.type == pygame.QUIT:`
`  running = False`
 	
`  screen.fill(WHITE)`
`  pygame.display.flip()`
`  clock.tick(60)`
 	
`pygame.quit()`
	The standard structure: handle events, update the game, draw the frame, refresh the display, and limit to 60 frames per second.

3. Events & keyboard input

SYNTAX / EXAMPLE	WHAT IT DOES
`event.type == pygame.QUIT`	The player clicked the window's close button.
`event.type == pygame.KEYDOWN` `event.type == pygame.KEYUP`	A key was pressed / released.

`event.key == pygame.K_SPACE`
`pygame.K_LEFT  pygame.K_RIGHT`
`pygame.K_UP  pygame.K_DOWN`
`pygame.K_a  pygame.K_d`
	Check which key was pressed. Common key constants.

`keys = pygame.key.get_pressed()`
`if keys[pygame.K_RIGHT]:`
`  x += speed`
	Continuous input: useful for smooth movement while a key is held down.

`event.type == pygame.MOUSEBUTTONDOWN` `event.pos`	Detect a mouse click and get its (x, y) position.

4. Drawing shapes

SYNTAX / EXAMPLE	WHAT IT DOES
`screen.fill((R, G, B))`	Fill the entire screen with a color.
`pygame.draw.rect(screen, RED,` `  (x, y, width, height))`	Draw a filled rectangle. Add a final argument such as 3 for a 3-pixel outline.

SYNTAX / EXAMPLE	WHAT IT DOES
`pygame.draw.circle(screen, BLUE, (center_x, center_y), radius)`	Draw a filled circle. Add a final argument for outline thickness.
`pygame.draw.line(screen, BLACK, (x1, y1), (x2, y2), 4)`	Draw a line between two points with 4-pixel thickness.
`pygame.draw.polygon(screen, GREEN, [(x1,y1), (x2,y2), (x3,y3)])`	Draw a filled polygon using a list of points.

PYGAME CHEAT SHEET

Movement • collisions • images • text • sound

5. Movement & the screen coordinate system

SYNTAX / EXAMPLE	WHAT IT DOES

`x += speed  # move right`
`x -= speed  # move left`
`y += speed  # move down`
`y -= speed  # move up`
	The origin (0, 0) is at the top-left. X increases to the right; Y increases downward.

`player_rect = pygame.Rect(x, y, 50, 50)`
`player_rect.x += 5`
`player_rect.y -= 5`
	A Rect stores position and size. It is useful for movement and collision detection.

`player_rect.center = (400, 300)`
`player_rect.topleft = (10, 10)`
`player_rect.right`
`player_rect.bottom`
	Common Rect properties for positioning and reading edges.

`dt = clock.tick(60) / 1000` `x += speed * dt`	Delta time (seconds) makes movement more consistent across different frame rates.

6. Collision detection

SYNTAX / EXAMPLE	WHAT IT DOES
`player_rect.colliderect(enemy_rect)`	Returns True if two rectangles overlap.
`player_rect.collidepoint(mouse_x, mouse_y)`	Returns True if a point is inside a rectangle.
`if player_rect.colliderect(enemy_rect):` `  print("Collision!")`	Run code when the player touches an enemy.

7. Images & text

SYNTAX / EXAMPLE	WHAT IT DOES

`player_img = pygame.image.load(`
`  "player.png").convert_alpha()`
`screen.blit(player_img, (x, y))`
	Load an image with transparency and draw it at (x, y). Keep the image file in the correct project path.

`small_img = pygame.transform.scale(` `  player_img, (64, 64))`	Resize an image to 64 × 64 pixels.

`font = pygame.font.Font(None, 36)`
`label = font.render("Score: 0", True, BLACK)`
`screen.blit(label, (10, 10))`
	Create and draw text. The True argument enables anti-aliasing.

8. Sound & useful utilities

SYNTAX / EXAMPLE	WHAT IT DOES

`pygame.mixer.init()`
`sound = pygame.mixer.Sound("jump.wav")`
`sound.play()`
	Load and play a short sound effect.

`pygame.mixer.music.load("music.mp3")`
`pygame.mixer.music.play(-1)`
`pygame.mixer.music.stop()`
	Play background music repeatedly (-1 means loop forever), then stop it.

`pygame.time.delay(1000)`	Pause for about 1000 milliseconds. Avoid using long delays inside the game loop.
`pygame.display.update()` `pygame.display.flip()`	Update the display. flip() refreshes the whole display; update() can refresh all or selected areas.

9. A minimal working template

SYNTAX / EXAMPLE	WHAT IT DOES

`import pygame`
 	
`pygame.init()`
`screen = pygame.display.set_mode((800, 600))`
`pygame.display.set_caption("My First Game")`
`clock = pygame.time.Clock()`
 	
`WHITE = (255, 255, 255)`
`x, y = 100, 100`
`speed = 5`
`running = True`
 	
`while running:`
`  for event in pygame.event.get():`
`  if event.type == pygame.QUIT:`
`  running = False`
 	
`  keys = pygame.key.get_pressed()`
`  if keys[pygame.K_LEFT]:`
`  x -= speed`
`  if keys[pygame.K_RIGHT]:`
`  x += speed`
 	
`  screen.fill(WHITE)`
`  pygame.draw.rect(screen, (30, 120, 220), (x, y, 50, 50))`
`  pygame.display.flip()`
`  clock.tick(60)`
 	
`pygame.quit()`
	Copy this template to start a simple game. Add event handling, game updates, and drawing inside the while loop.

Remember: The usual order in each frame is **Events** → **Update** → **Draw** → **Display** → **Tick**. Call `pygame.display.flip()` after drawing so the player can see the new frame.